Celery Task Conversion

```
from pydantic import BaseModel, model_validator
from pydantic.alias_generators import to_camel,
to_snake, to_pascal
from uuid import UUID, uuid4
from typing import Optional
from typing_extensions import Self
from enum import Enum

class RoleEnum(Enum):
    ADMIN = "admin"
    DEVELOPER = "developer"
    MAINTAINER = "maintainer"
    VIEWER = "viewer"

class User(BaseModel):
    id: UUID
    name: str
    password: str
    password_repeat: str
    age: int
    email: str
    phone_no: str
    role: Optional[RoleEnum]

    @model_validator(mode='after')
    def check_passwords(self) -> Self:
        if self.password != self.password_repeat:
            raise ValueError("Passwords dont match")
        return self

    @model_validator(mode='after')
    def is_email(self) -> Self:
        if "@" not in self.email:
            raise ValueError("Enter valid email")
        return self

user_dict = {
    "id": uuid4(),
    "name": "AdityaZala",
```

```
    "password": "Aditya3919",
    "password_repeat": "Aditya3919",
    "age": 21,
    "email": "adityazala@gmail.com",
    "phone_no": "1234567890",
    "role": "admin"
}

parsed = User.model_validate(user_dict)
output = parsed.model_dump()

print(output)
```

in this code i have used model validator. but in reality it only validates fields. like password and email. so shouldnt it be called field validator?

Good question—and you're noticing something subtle but important.

At first glance, yes, it *looks* like you're validating individual fields (`password`, `email`). But what matters is **what data the validator has access to and what it's allowed to reason about**.

🧠 The real distinction

🔹 Field validator

- Runs **per field**
- Sees **only that field's value** (plus limited context)
- Use when validation is **independent**

🔹 Model validator

- Runs on the **whole model**
- Has access to **all fields together**
- Use when validation depends on **relationships or multiple fields**

🔍 Let's analyze your validators

1. Password check

```python
@model_validator mode='after'
def check_passwords self      Self
    if self.              != self.
        raise ValueError "Passwords dont match"
```

👉 This is **definitely a model validator**

Why?

Because:

- It compares **two fields**
- A field validator **cannot safely do this**

---

2. Email check

```python
@model_validator mode='after'
def is_email self      Self
    if "@" not in self.
        raise ValueError "Enter valid email"
```

👉 This one *looks* like a field validator

Because:

- It only uses `self.email`
- No dependency on other fields

🤔 So your confusion is valid

You're thinking:

> "If it only checks one field, why not field validator?"

And the answer is:

> You *can* use a field validator here—but you don't *have* to.

⚖️ When to prefer each (practical rule)

✅ Use `@field_validator` when:

- Logic is **purely field-specific**
- You want validation to happen **earlier**
- You want better **error localization**

Example:

```python
from pydantic import field_validator

@field_validator "email"
@classmethod
def validate_email cls  v
    if "@" not in v
        raise ValueError "Enter valid email"
    return v
```

✅ Use `@model_validator` when:

- You need **multiple fields**
- You need the **final validated model**
- You care about **cross-field consistency**

⚠ Important difference in execution timing

Your current validators are:

```python
mode='after'
```

So flow is:

1. All fields validated individually
2. Model created
3. `check_passwords()` runs
4. `is_email()` runs

If email was a field validator:

```python
```

```
@field_validator "email"
```

Then:

- It runs **during field validation phase**
- Before model is fully built

🧩 Subtle but powerful insight

Even though your `is_email` looks like field validation, using `model_validator` means:

- It runs **after transformations**
- It sees **final processed values**
- It behaves consistently with other model-level rules

🚀 Clean version (best practice)

```python
from pydantic import field_validator

@field_validator "email"
@classmethod
def validate_email cls  v
    if "@" not in v
        raise ValueError "Enter valid email"
    return v
```

Keep this:

```python
@model_validator mode='after'
def check_passwords self      Self
    if self.              != self.
        raise ValueError "Passwords dont match"
    return self
```

🧠 Final mental model

- Field validator → **“Is this value valid?”**
- Model validator → **“Do these values make sense together?”**

💡 One more thing (expert-level hint)

If tomorrow you add:

```python
confirm_email
```

Then your email validator might **move to model level**, because now it's cross-field.

So your instinct is right—but the naming is about **scope**, not just what you happened to check in that function.

That conclusion is **mostly right**, but it's a bit too narrow—and that's where it can trip you up later.

Let's refine it so it holds even in edge cases.

✅ Your current conclusion

> "model_validator is used when more than 1 fields have to be compared or whole model has to be validated"

✔️ This is **correct in spirit**
❌ But **incomplete**

🧠 What's missing?

A `model_validator` is not *defined* by "number of fields used".

It's defined by **the level at which validation happens**.

🔍 Better definition (more accurate)

> **Model validator = runs at the model level, with access to the entire dataset (raw or processed), regardless of how many fields you actually use.**

🔄 Why your definition can break

Consider this:

```python
@model_validator mode='before'
def normalize_input cls  data
    data "email"  = data "email" .        .
    return data
```

👉 This is a **model validator**, but:

- It only touches **one field**
- No comparison involved

So your rule would incorrectly classify it as a field validator.

⚖️ Correct way to think

Instead of:

> "How many fields am I using?"

Think:

> "At what stage and scope do I need control?"

🎯 Clean mental model

🔹 **Use `field_validator` when:**

- You validate **a single field in isolation**
- Logic depends only on that field's value

👉 Example:

```python
email must contain "@"
age must be > 18
```

🔹 **Use `model_validator` when:**

1. Cross-field logic (your case)

```python
password == password_repeat
```

2. Pre-processing input (VERY important use-case)

```python
mode='before'
```

- Raw input cleanup
- Key transformation
- Merging/splitting fields

---

3. Post-processing / enforcing invariants

```python
mode='after'
```

- Final consistency checks
- Derived values
- Business rules

🧩 Reframed conclusion (strong version)

Here's a version that won't fail you later:

> **Use `model_validator` when you need control over the entire model lifecycle (before or after validation), especially for cross-field logic or input transformation.**

⚠️ Why this matters (real-world impact)

If you stick to your original rule, you'll miss:

- input normalization problems
- alias/key transformation issues
- complex API payload handling
- pre-validation fixes (super common in production)

🔥 Tie back to your learning path

You're currently learning:

- aliasing
- validation flow
- model lifecycle

👉 The *real power* of `model_validator` shows up in:

```python
mode='before'
```

That's where:

- messy API input gets cleaned
- snake_case / camelCase gets unified
- defaults get injected dynamically

🚀 Final takeaway

Your version:

> "model_validator = multi-field validation"

Better version:

> "model_validator = model-level control (before/after), often used for cross-field logic but not limited to it"

If you want, next step I'd strongly recommend:
👉 Walk through a **before → field → after execution trace step-by-step**

That's where everything clicks permanently.

```
@model_validator(mode='before')
    def correct_phone_no(cls, data):
        data["phone_no"] = "+91-"+data["phone_no"]
        return data
```

is this correct?

You're on the right track using a `mode='before'` **model validator** for input transformation—but this exact implementation is a bit fragile. It'll work in the happy path, but break in real scenarios.

Let's tighten it.

⚠️ Problems in your current code

```python
@model_validator mode='before'
def correct_phone_no cls  data
    data "phone_no"  = "+91-"+data "phone_no"
    return data
```

1. ❌ Assumes `data` is always a dict

In `mode='before'`, `data` can sometimes be:

- dict ✅ (most common)
- model instance ❗
- other types (rare but possible)

---

2. ❌ Assumes `phone_no` always exists

If input is missing it:

```python
```

👉 This will crash with `KeyError`

---

3. ❌ Blindly prepends `+91-`

If user already sends:

```python
```

```
"+91-1234567890"
```

👉 You'll end up with:

```
[redacted]
```

✅ Robust version (production-safe)

```python
from typing import Any

@model_validator mode='before'
@classmethod
def correct_phone_no cls  data  Any
    if not isinstance data  dict
        return data

    phone = data.      "phone_no"
    if phone and not phone.                "+91-"
        data "phone_no"  = f"+91-  phone  "

    return data
```

🧠 What's happening here (important)

- `mode='before'` → runs **before field validation**
- You are modifying **raw input**
- This is exactly the **right place** for normalization like:
  - formatting
  - key conversion
  - default injection

🔄 Execution flow (your case)

1. Input comes in:

```python
 "phone_no"  "1234567890"
```

2. `before` validator runs:

```python
"phone_no"  "+91-1234567890"
```

3. Field validation happens:

```python
phone_no  str ✅
```

4. Model is created

⚖️ Should this be field_validator instead?

You *could* do:

```python
@field_validator "phone_no"  mode="before"
@classmethod
def format_phone cls  v
    if not v.                "+91-"
        return f"+91- v "
    return v
```

Difference:

Approach	Scope	When to use
model_validator(before)	full input	multi-field or key normalization
field_validator(before)	single field	cleaner for isolated transformation

👉 For this case, **field_validator is actually cleaner**

🎯 Final recommendation

- Your approach is **conceptually correct**
- But for this specific logic → prefer **field_validator**

🚀 Key takeaway

> Use `model_validator(mode='before')` when you need to reshape the **entire input**
> Use `field_validator(mode='before')` when you're just fixing **one field**

If you want next, I can show:

- how `before` behaves with alias (`PhoneNo` vs `phone_no`)
- or a full trace combining alias + before + after validators (that's where things get really interesting)

where to use validation error and where to use value error in pydantic??

This is a really important distinction—and many people get it slightly wrong at first.

Let's make it crisp and practical.

🧠 Short Answer

- ✅ **Raise `ValueError` (or `TypeError`) inside validators**
- ❌ **Do NOT raise `ValidationError` manually**

🔥 Why?

1. `ValueError` → what YOU raise

Inside validators:

```python
from pydantic import field_validator

@field_validator "email"
@classmethod
def validate_email cls  v
    if "@" not in v
        raise ValueError "Invalid email"
    return v
```

👉 You raise `ValueError`

---

2. `ValidationError` → what PYDANTIC raises

When validation fails, Pydantic collects all errors and raises:

```python
from pydantic import ValidationError
```

👉 This is a **wrapper error** that contains:

- field errors
- locations
- messages
- types

Example:

```python
try
    User.                    data
except ValidationError as e
    print e
```

⚠ Why you should NOT raise `ValidationError`

If you try this:

```python
from pydantic import ValidationError

raise ValidationError
```

👉 You'll:

- bypass Pydantic's error system
- break error formatting
- lose automatic context (field name, location, etc.)
- possibly crash incorrectly

🧩 Mental Model

Think like this:

- You → signal a problem (`ValueError`)
- Pydantic → collects + structures → raises `ValidationError`

🔍 What happens internally

1. Your validator runs
2. You raise:

```python
ValueError "Invalid email"
```

3. Pydantic catches it and converts to:

[redacted]

✅ When to use what

✔️ Use `ValueError` when:

- validating values
- business rules fail
- custom checks fail

✔️ Use `TypeError` when:

- wrong type (rare, Pydantic usually handles this)

✔️ Catch `ValidationError` when:

- calling `.model_validate()`
- handling API input errors
- returning errors in FastAPI

🚀 Example end-to-end

```python
try
    user = User.                    data
except ValidationError as e
    print e.
```

⚠️ Advanced note (expert-level)

If you want more structured/custom errors, you can use:

```python
from pydantic_core import PydanticCustomError
```

But even then:
👉 You still **don't raise ValidationError manually**

🎯 Final takeaway

> **Raise simple errors (`ValueError`) → let Pydantic build the complex `ValidationError`.**

If you want next level:

- how error locations (`loc`) are built
- or how multiple validators combine into one `ValidationError`

That's where debugging becomes very easy.